EPIC 05 – SERVICE COMMUNICATION & RESILIENCE

Task 02: Timeout & Retry

The task requires:

- Connection timeout
- Read timeout
- Retry
- Retry limits

Since **Task 01 uses RestClient**, we can add timeout and retry to the same REST communication.

Step 1 — Add dependencies

Open your **Order Service → pom.xml**.

Add:

```
<!-- Apache HttpClient 5 -->
<dependency>
  <groupId>org.apache.httpcomponents.client5</groupId>
  <artifactId>httpclient5</artifactId>
</dependency>

<!-- Resilience4j Retry -->
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>

<!-- Spring Boot Actuator -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Save pom.xml and wait for Maven to download the dependencies.

Step 2 — Configure connection and read timeout

Create:

src/main/java

└── com.example.orderservice

└── config

└── RestClientConfig.java

Add,

```
package com.example.orderservice.config;

import java.time.Duration;

import org.apache.hc.client5.http.config.RequestConfig;
import org.apache.hc.client5.http.impl.classic.CloseableHttpClient;
import org.apache.hc.client5.http.impl.classic.HttpClients;
import org.apache.hc.client5.http.impl.io.PoolingHttpClientConnectionManagerBuilder;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.HttpComponentsClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

@Configuration
public class RestClientConfig {

  @Bean
  public RestClient restClient(RestClient.Builder builder) {

    RequestConfig requestConfig = RequestConfig.custom()
        .setConnectTimeout(3000)
```

```
            .setResponseTimeout(5000)
            .build();

        CloseableHttpClient httpClient = HttpClients.custom()
            .setDefaultRequestConfig(requestConfig)
            .setConnectionManager(
                PoolingHttpClientConnectionManagerBuilder.create().build()
            )
            .build();

        HttpComponentsClientHttpRequestFactory factory =
            new HttpComponentsClientHttpRequestFactory(httpClient);

        return builder
            .requestFactory(factory)
            .baseUrl("http://localhost:8082")
            .build();
    }
}
```

Here:

Connection timeout = 3 seconds
Read/response timeout = 5 seconds

So if Product Service cannot establish a connection within 3 seconds, the request fails.

If the connection is established but Product Service does not respond within 5 seconds, the request fails.

Step 3 — Configure Retry

Open:

src/main/resources/application.properties

Add:

resilience4j.retry.instances.productService.max-attempts=3

resilience4j.retry.instances.productService.wait-duration=2s

resilience4j.retry.instances.productService.retry-exceptions=org.springframework.web.client.ResourceAccessException,org.springframework.web.client.HttpServerErrorException

This means:

Maximum attempts = 3
Wait between attempts = 2 seconds

So:

Attempt 1

↓

Failure

↓

Wait 2 seconds

↓

Attempt 2

↓

Failure

↓

Wait 2 seconds

↓

Attempt 3

↓

Success / Final failure

Step 4 — Add @Retry

Open your existing:

ProductClient.java

Use:

```
package com.example.orderservice.client;

import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

import io.github.resilience4j.retry.annotation.Retry;

@Component
public class ProductClient {

   private final RestClient restClient;

   public ProductClient(RestClient restClient) {
      this.restClient = restClient;
   }

   @Retry(name = "productService")
   public ProductResponse getProductById(Long productId) {

      return restClient.get()
            .uri("/products/{id}", productId)
            .retrieve()
            .body(ProductResponse.class);
   }
}
```

Notice that we are now injecting the `RestClient` created in `RestClientConfig`.

Step 5 — Test the timeout

Start your:

Order Service → 8081
Product Service → 8082

Call:

GET http://localhost:8081/orders/product/1

Normally:

Order Service

|

| RestClient

↓

Product Service

|

↓

Response

If Product Service becomes unavailable:

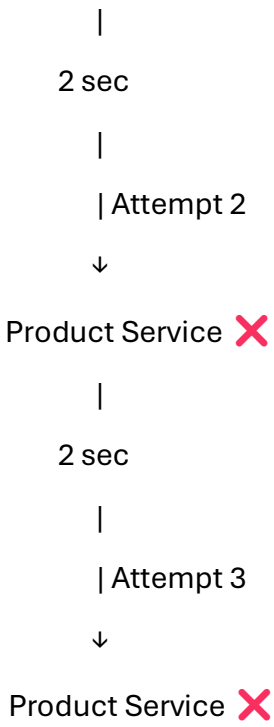
Order Service

|

| Attempt 1

↓

Product Service ❌



After the third attempt, the request fails instead of retrying forever.

Task 02 Result

You have implemented all four requirements:

Requirement	Implementation
Connection timeout	`setConnectTimeout(3000)`
Read/response timeout	`setResponseTimeout(5000)`
Retry	`@Retry(name = "productService")`
Retry limits	`max-attempts=3`

Final project structure:

```
Order Service
├── config
│       └── RestClientConfig.java
├── client
│       ├── ProductClient.java
│       └── ProductResponse.java
├── service
│       └── OrderService.java
├── controller
│       └── OrderController.java
│
└── resources
        └── application.properties
```

This completes **Task 02 — Timeout & Retry.**